PLATAFORMA DE ESTUDO COM FLASHCARDS

Nex_TI

Estudar TI com cartas que voltam na hora certa.

DevTime

Gabriel Alves Moreira (H67HJI4) · Maciel Costa da Silva (R280985) · Maycon Douglas Inácio Silva (H719CD3)

Miguel Angel Fernandez Ortiz (H7858F9) · Rafael Mesquita (H6722I0)

github.com/MayconDIS/PIM_IV

Um app para estudar TI um pouco por dia

O aluno estuda por flashcards, e o sistema decide quando cada carta volta.

Flashcards com SM-2

Errou, a carta volta amanhã. Acertou, ela demora mais para voltar.

Trilha de estudo

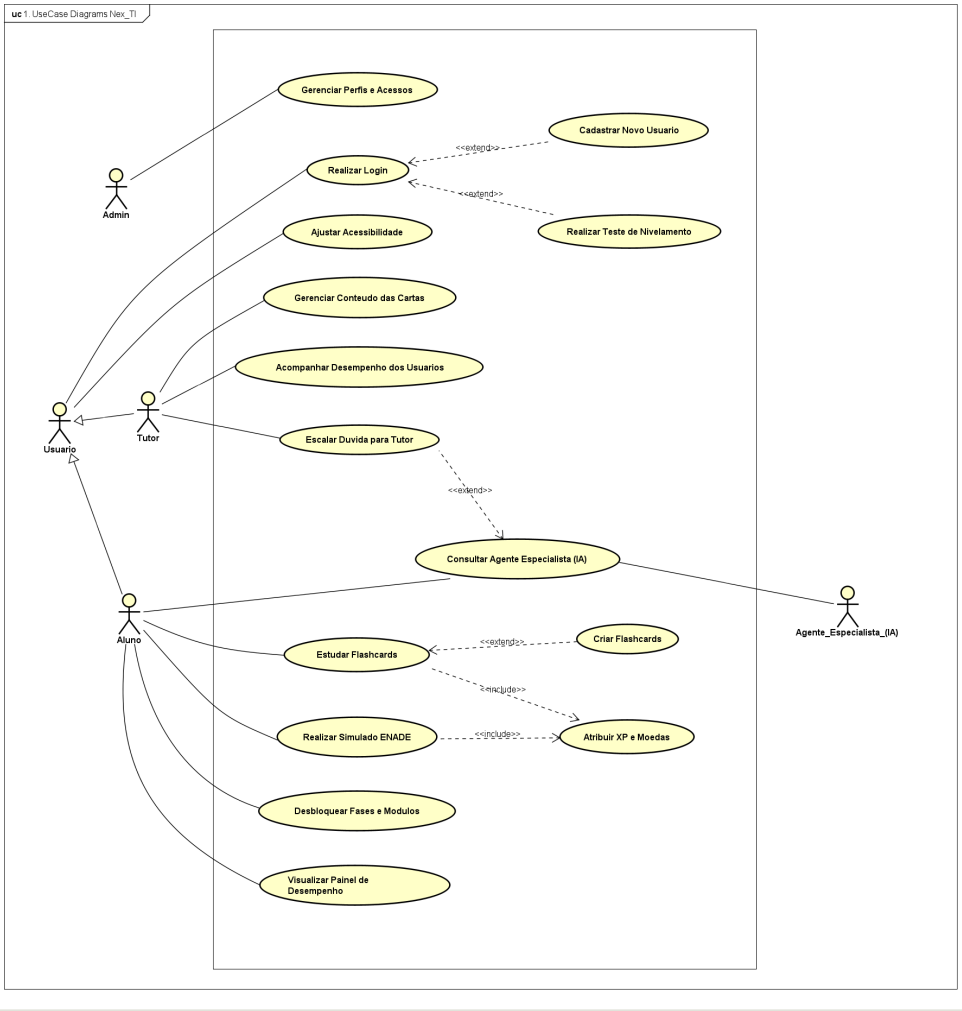
26 fases e 620 cartas sobre TI, liberadas em ordem. No fim, simulado no estilo ENADE.

XP e moedas

Cada fase dá XP e moedas. As moedas liberam disciplinas bônus.

O PIM IV continua o PIM III: mesmo projeto, agora com a API em camadas, app de celular e nuvem.

Quem usa o sistema e para quê

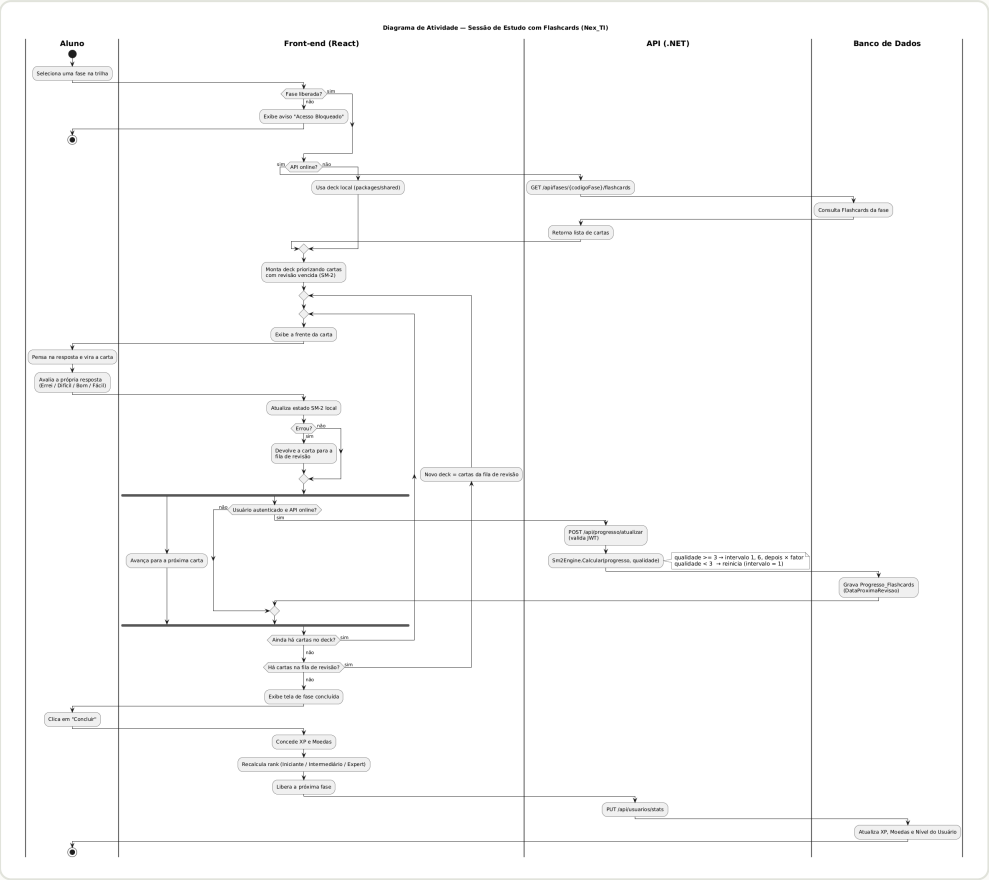


Aluno: estuda os flashcards, faz simulados e vê o próprio desempenho.

Tutor: cria conteúdo e acompanha os alunos.

Admin: gerencia quem tem acesso.

Uma sessão de estudo, passo a passo



1. O aluno escolhe uma fase. Se estiver bloqueada, aparece um aviso.
2. O site busca as cartas na API. Sem internet, usa as cartas guardadas.
3. O aluno responde e o SM-2 calcula quando a carta volta.
4. Errou? A carta volta no fim da sessão.
5. Acabou: ganha XP e moedas e libera a próxima fase.

Onde a POO aparece no Nex_TI

Herança

Aluno e Tutor herdam login, XP e moedas de Usuario.

```
class Aluno : Usuario
```

Interface

ISm2Engine diz o que o SM-2 faz; Sm2Engine é quem faz.

```
class Sm2Engine : ISm2Engine
```

Polimorfismo

A carga inicial do banco roda o conteúdo e os simulados pelo mesmo método, sem saber qual é qual.

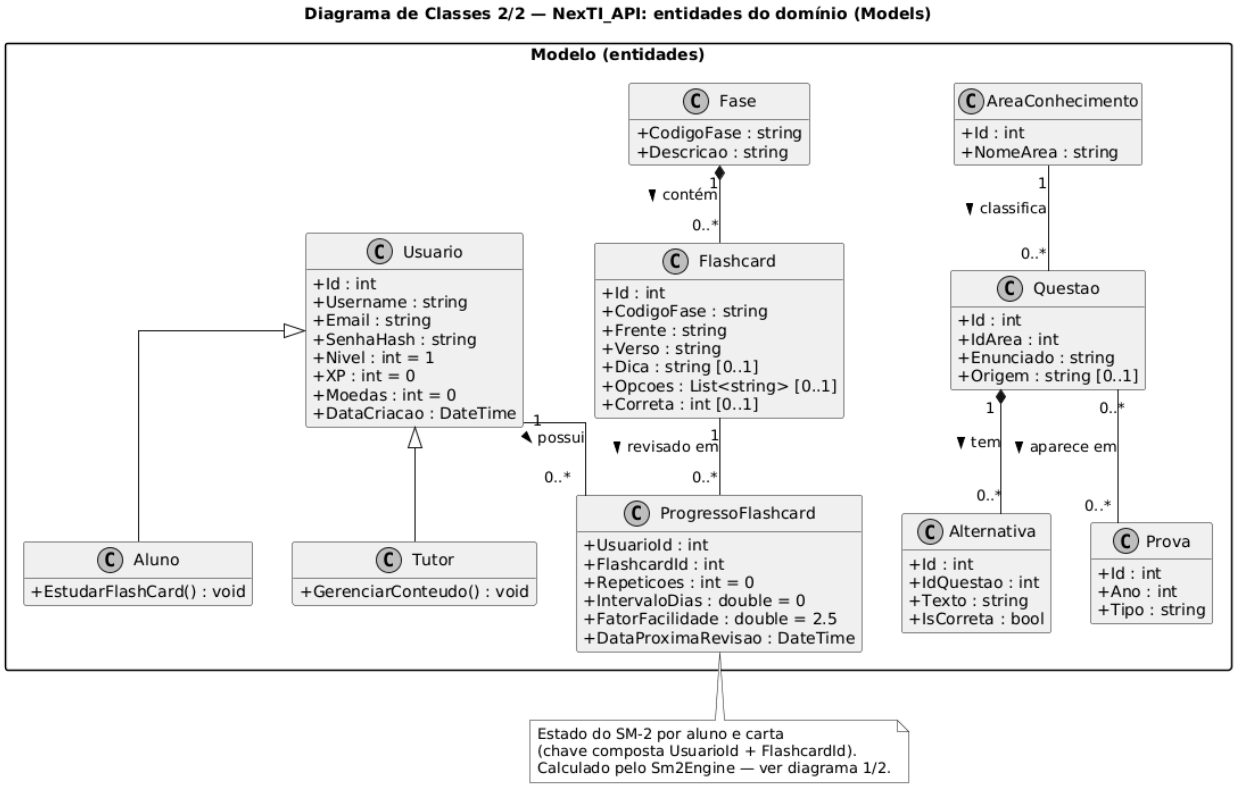
```
etapa.ExecutarAsync(db)
```

Encapsulamento

A regra do XP fica dentro do serviço: só sobe, e no máximo 200 por vez.

```
GamificacaoService.ValidarAtualizacao()
```

As classes que estão no código



Usuario é a classe pai de Aluno e Tutor.

ProgressoFlashcard guarda o SM-2 de cada aluno em cada carta.

Novas no PIM IV: Prova, Questao, Alternativa e AreaConhecimento, para os simulados.

Cada parte da API cuida de uma coisa

Rotas

Recebem o pedido e devolvem a resposta.

`Endpoints/`

Serviços

As regras: login, SM-2, XP e conteúdo.

`Services/`

Dados

Conversam com o banco SQL Server.

`Data/`

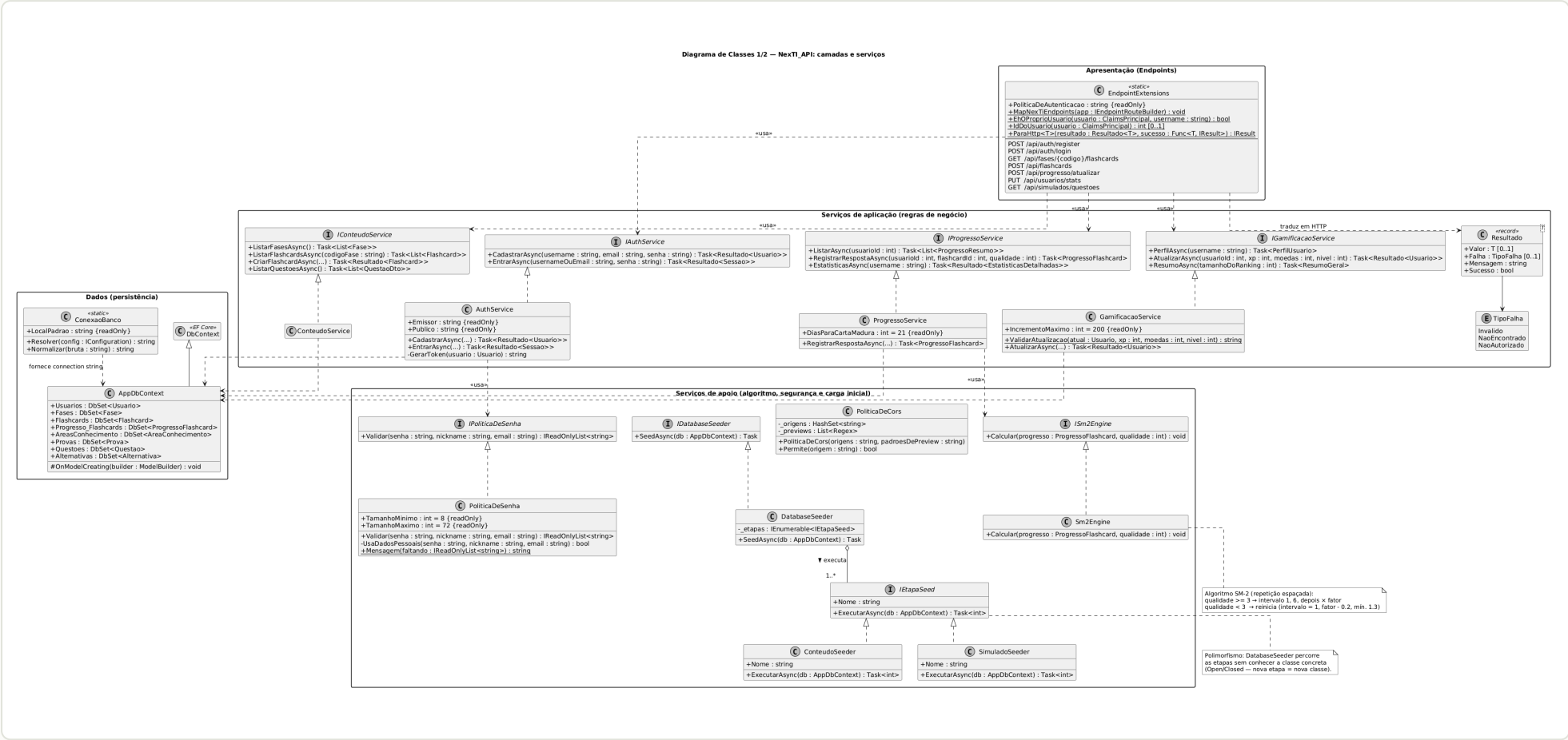
Modelos

As classes: Usuario, Flashcard, Fase...

`Models/`

Antes: as rotas faziam tudo, num arquivo de 338 linhas. **Agora:** 142 linhas, e só os serviços mexem no banco.

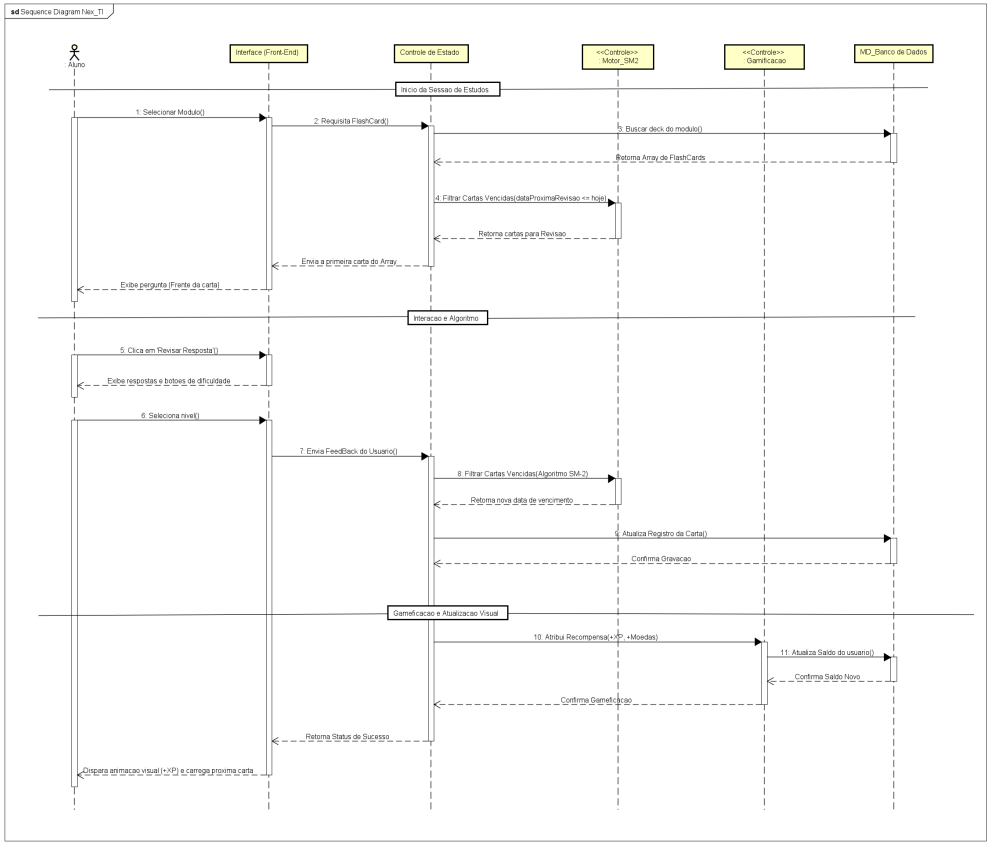
As camadas no diagrama de classes



Um repositório, cada parte na sua pasta

PASTA	O QUE TEM
`sistema/apps/api`	API em C# (.NET 10)
`sistema/apps/react-web`	O site, em React
`sistema/apps/react-mobile`	O app de celular
`sistema/packages/shared`	Cartas e código usados pelo site e pelo app
`sistema/tests`	62 testes automáticos da API
`relatorio/`	A monografia (ABNT)
`modelagem-uml/`	Os diagramas (Astah e PlantUML)

O SM-2 em ação



O aluno dá uma nota de 0 a 5 para a própria resposta.

Nota 3 ou mais

Volta em 1 dia, depois em 6, depois cada vez mais longe.

Nota menor que 3

Volta amanhã e recomeça a contagem.

```
Sm2Engine.Calcular()
```

Como o código chega na nuvem

62
testes automáticos

14
rotas na API

22
itens no backlog

5
sprints no Trello

Pull Request → **Merge na main** → **Testes rodam** → **API na Azure, site na Vercel**

Obrigado!

Perguntas?

github.com/MayconDIS/PIM_IV